

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
Институт информатики и кибернетики
Кафедра технической кибернетики

Отчет по лабораторной работе №0
Дисциплина: «Технологии сетевого программирования»

Тема «Выбор и описание предметной области. Проектирование функционала
приложения»

Выполнили: Берестов И.М.

Лучин М.Д.

Группа: 6405-010302D

Преподаватель: Головастиков Н.В.

Самара, 2026

СОДЕРЖАНИЕ

1 Бизнес цель и предметная область	3
1.1 Описание предметной области	3
1.2 Бизнес-цель проекта	3
2 Функционал приложения и сценарии использования	4
2.1 Функционал	4
2.2 Сценарии использования	4
3 Архитектура приложения и стек технологий	6
4 Структура базы данных	8

1 Бизнес цель и предметная область

1.1 Описание предметной области

Система представляет собой платформу социальной сети, где пользователи могут публиковать посты, подписываться на авторов, просматривать ленту новостей и ставить оценку. Система оперативно доставляет автору уведомления о новых оценках. Администраторы платформы имеют возможность модерировать контент и блокировать нарушителей.

1.2 Бизнес-цель проекта

Создать отказоустойчивое и масштабируемое веб-приложение для публикации контента и сбора обратной связи от пользователей.

2 Функционал приложения и сценарии использования

2.1 Функционал

Пользователь:

- Регистрация и авторизация в системе.
- Создание, просмотр и удаление собственных постов.
- Подписка/отписка от других пользователей.
- Просмотр ленты подписок и постов.
- Выставление и изменение оценки и короткого отзыва к посту другого пользователя.
- Получение уведомлений в реальном времени.

Администратор:

- Просмотр списка всех пользователей.
- Блокировка и разблокировка аккаунтов пользователей.
- Удаление любых постов и оценок, нарушающих правила платформы.

2.2 Сценарии использования

Сценарий 1: Регистрация и публикация поста

1. Пользователь отправляет POST-запрос на регистрацию с логином и паролем.
2. После успешной регистрации выполняет вход, получает JWT-токен.
3. Отправляет запрос на создание поста.
4. Пост сохраняется в БД.

Сценарий 2: Просмотр ленты и рекомендации

1. Пользователь переходит на страницу ленты.

2. API-сервис запрашивает у gRPC-сервиса список рекомендованных постов на основе подписок пользователя и постов с наивысшим средним рейтингом.
3. Пользователь получает отсортированную ленту.

Сценарий 3: Выставление оценки посту и мгновенное уведомление

1. Пользователь оценивает пост другого пользователя на n баллов с комментарием.
2. API-сервис сохраняет оценку в БД и отправляет событие о новой оценке в RabbitMQ.
3. Сервис уведомлений считывает событие из RabbitMQ и по WebSocket отправляет автору поста сообщение: "Пользователь X поставил вашему посту n/m ".

3 Архитектура приложения и стек технологий

Приложение спроектировано в виде распределённой микросервисной системы, разворачиваемой в Docker. Архитектура разделена на три основных слоя:

Сервисный слой:

- FastAPI Service является единой точкой входа для пользовательских запросов по протоколу REST API.
- Auth Service отвечает за аутентификацию и авторизацию пользователей с использованием JWT-токенов.

Микросервисы и брокер сообщений:

- gRPC Service отвечает за ресурсоёмкие операции - формирование персональной ленты новостей и алгоритм рекомендации. Взаимодействие с FastAPI Service происходит синхронно по протоколу gRPC.
- RabbitMQ Server используется как асинхронный брокер сообщений. При создании пользователем событий, FastAPI Service публикует событие в очередь.
- Socket Service подписывается на события из RabbitMQ и доставляет уведомления подключённым клиентам в режиме реального времени через постоянное WebSocket-соединение.

Слой данных:

- DB Service реализует бизнес-логику работы с базой данных через SQLAlchemy.
- PostgreSQL выступает в качестве основного хранилища данных.

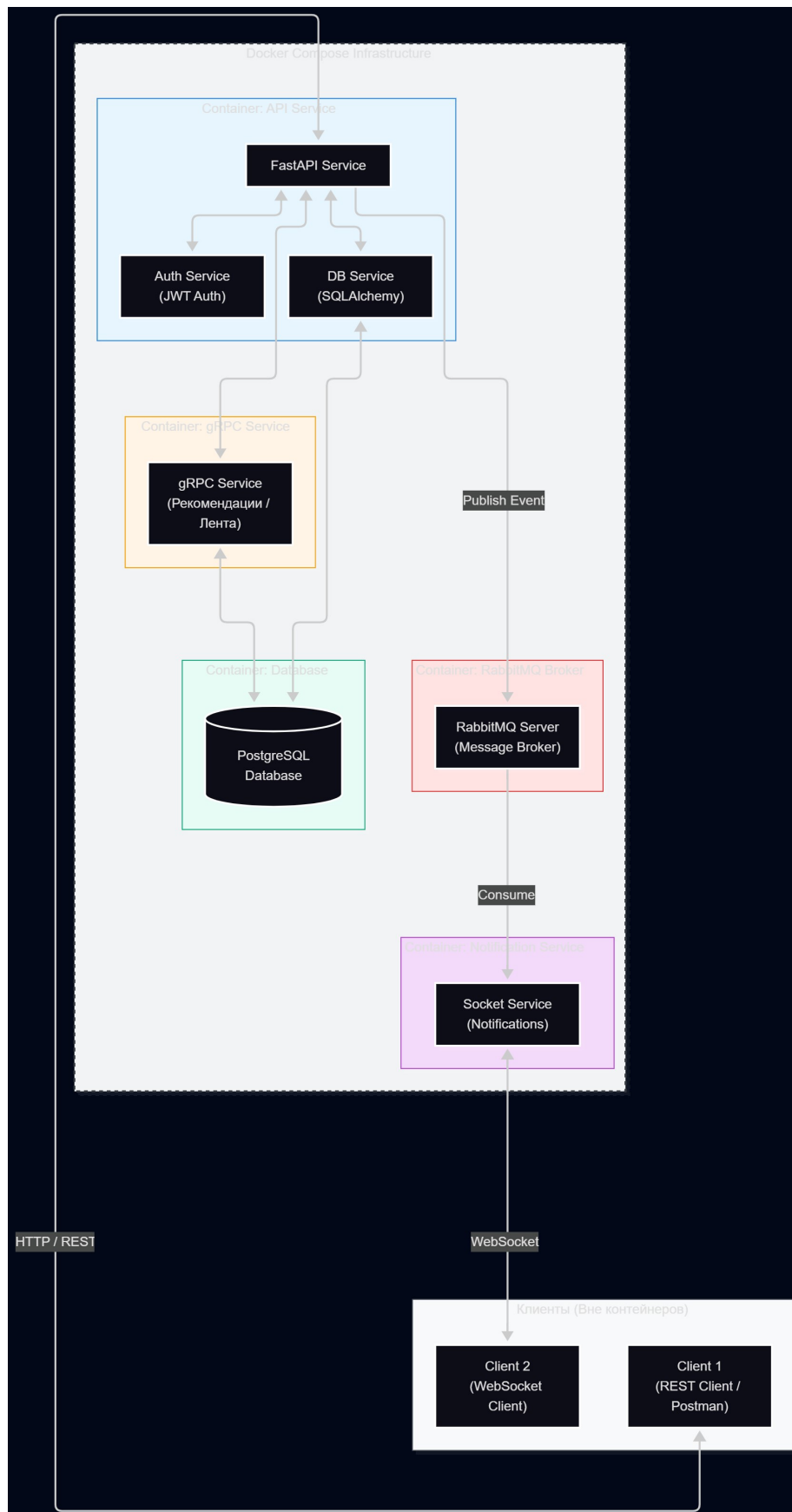


Рисунок 1 – Архитектура приложения

4 Структура базы данных

USER (Пользователи):

Хранит ключевые данные аккаунта:

- идентификатор (id),
- имя (username),
- системную роль (ROLE)
- статус блокировки (is_banned).

POST (Публикации):

Содержит контент постов (content). Связана с таблицей USER связью «один-ко-многим» через внешний ключ author_id.

FOLLOWER (Подписки):

Реализует связь «многие-ко-многим» между пользователями. Состоит из составного первичного ключа (follower_id, followed_id), где оба поля являются внешними ключами к USER.

POST_RATINGS (Оценки постов):

Реализует связь «многие-ко-многим» между пользователями и постами. Хранит рейтинг (score) и текстовый отзыв (review_text). Составной первичный ключ (user_id, post_id) гарантирует, что один пользователь может оставить только одну оценку конкретному посту.

USER_PASSWORD:

Хранит id хозяина пользователя и закодированный hash пароля.

ROLE:

Системная роль, выдающаяся пользователю

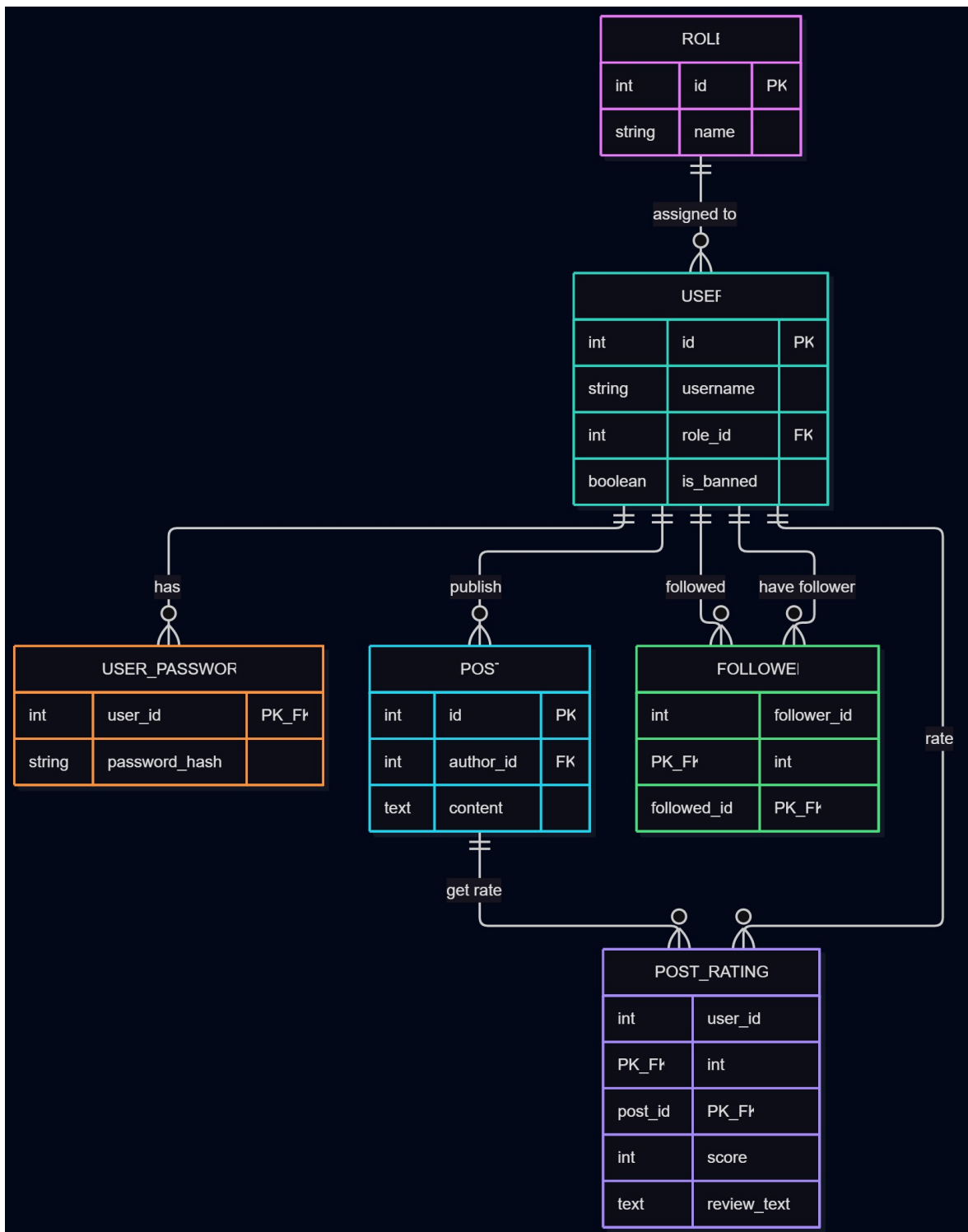


Рисунок 2 – ER диаграмма БД